

JavaScript Object Prototypes

[Back to JS page](#)

Table of Contents

- [JavaScript Object Prototypes](#)
 - [Prototype Inheritance](#)
 - [Example 1](#)
 - [Adding Properties to Objects](#)
 - [Example 2](#)
 - [Adding Methods to Objects](#)
 - [Example 3](#)
 - [Using the prototype Property](#)
 - [Example 4](#)
 - [Document](#)
 - [Reference](#)

Prototype Inheritance

All JavaScript objects inherit properties and methods from a **prototype**.

- `Date` objects inherit from `Date.prototype`
- `Array` objects inherit from `Array.prototype`
- `Person` objects inherit from `Person.prototype`

The `Object.prototype` is on the top of the prototype chain:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Prototypes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Prototype Inheritance</h4>
<p id="demo"></p>

<script>
function Person(first, last, age) {
  this.firstName = first;
  this.lastName = last;
  this.age = age;
}

const myFather = new Person("John", "Doe", 50);
const myMother = new Person("Jane", "Doe", 48);

document.getElementById("demo").innerHTML =
  "My father is " + myFather.firstName + " " + myFather.lastName + "<br>" +
  "My mother is " + myMother.firstName + " " + myMother.lastName;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Adding Properties to Objects

You can add new properties to an existing object by simply giving it a value. However, this only adds it to that specific object, not to the prototype:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Prototypes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Properties to Objects</h4>
<p id="demo"></p>

<script>
function Person(first, last) {
  this.firstName = first;
  this.lastName = last;
}

const person1 = new Person("John", "Doe");
const person2 = new Person("Jane", "Smith");

// Add a property to person1 only
person1.nationality = "American";

document.getElementById("demo").innerHTML =
  "person1 nationality: " + person1.nationality + "<br>" +
  "person2 nationality: " + person2.nationality + " (undefined)";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Adding Methods to Objects

You can add a method to an existing object. But again, this only adds it to that specific object:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Prototypes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Methods to Objects</h4>
<p id="demo"></p>

<script>
function Person(first, last) {
  this.firstName = first;
  this.lastName = last;
}

const person1 = new Person("John", "Doe");
const person2 = new Person("Jane", "Smith");

// Add a method to person1 only
person1.fullName = function() {
  return this.firstName + " " + this.lastName;
};

document.getElementById("demo").innerHTML =
  "person1: " + person1.fullName() + "<br>" +
  "person2 would have an error if we called fullName()";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Using the prototype Property

The JavaScript `prototype` property allows you to add new properties and methods to **all** objects of a given type:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Object Prototypes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Using the prototype Property</h4>
<p id="demo"></p>

<script>
function Person(first, last) {
  this.firstName = first;
  this.lastName = last;
}

// Add a property to ALL Person objects using prototype
Person.prototype.nationality = "American";

// Add a method to ALL Person objects using prototype
Person.prototype.fullName = function() {
  return this.firstName + " " + this.lastName;
};

const person1 = new Person("John", "Doe");
const person2 = new Person("Jane", "Smith");

document.getElementById("demo").innerHTML =
  "person1: " + person1.fullName() + ", " + person1.nationality + "<br>" +
  "person2: " + person2.fullName() + ", " + person2.nationality;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Object Prototypes](#)